Lecture ~~8 10~~, DAA ~~Feb 13~~

Priority Queues : Min, Extract-min $O(1)$, $O(\log n)$
Insert / Delete $O(\log n)$

(Min) Heap : Efficient implementation of the above opns

Build heap (n) takes $O(n)$

Build BBST (n) takes $\Theta(n \log n)$

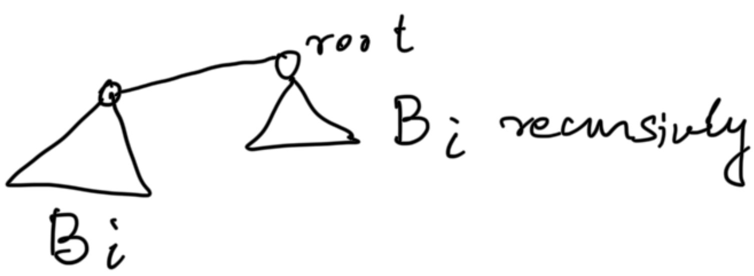
Give 2 heaps H_1 and H_2, we want to construct a heap $H = H_1 \cup H_2$

Merging / Melding heaps

We define a family of trees where

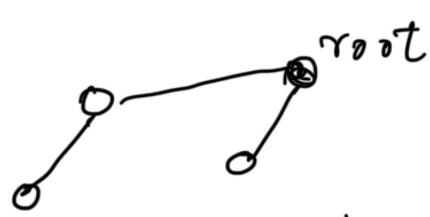
B_0 : • single node

B_{i+1}



B_1 =

B_2 -



B_3

Binomial Trees

Properties

(i) Number of nodes in B_i = 2^i nodes

(ii) Height of B_i = i

(iii) Degree of root node i

(iv) # nodes in level $j \le i$

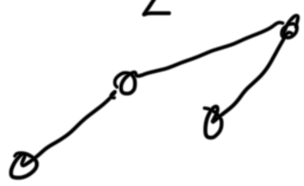
$$\binom{i}{j}$$

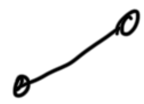
$\boxed{\text{What we do about a size } \cancel{\text{that}} \text{ is not } 2^i \text{ ?}}$

Idea: a $\underbrace{\text{binomial tree}}$ of 6 nodes

set of binomial trees such that the # nodes add up to 6

$2^2 = B_2$





$B_1 = 2^1$

In a set ~~of~~ Binomial trees, we are allowed at most one tree of B_i for any i

So for a number n, consider the binary representation

$$\overbrace{b_{k-1}\ b_{k-2}\ \cdots\ b_0}^{k}$$

$n = 11$

1 0 1 1

a circular list ordered on i's

Binomial Heap

above constraint

on n values is a set of heap ordered Binomial Trees

how many Binomial trees are needed

$\le \lceil \log n \rceil$

Given two Binomial Heaps how do we merge them?

BH_1

BH_2

$B_7 - B_6 - B_4 - B_2 - B_1 - B_0$ $\qquad$ $B_6 - B_5 - B_3 - B_1 - B_0$

These can be merged in $O(\log n)$ time with the required properties exactly analogous to adding two $\log n$ bit numbers